

The background is a light blue gradient with several realistic water droplets of various sizes scattered across the surface. The droplets have highlights and shadows, giving them a three-dimensional appearance.

SCALER-OPERATOR E PROMETHEUS OPERATOR

OBIETTIVO DEL LAVORO

Lo scopo di questo progetto è quello di sviluppare un operator per Kubernetes che permetta di applicare degli opportuni meccanismi di scaling e migrazione per la gestione di eventuali sovraccarichi dei pod.

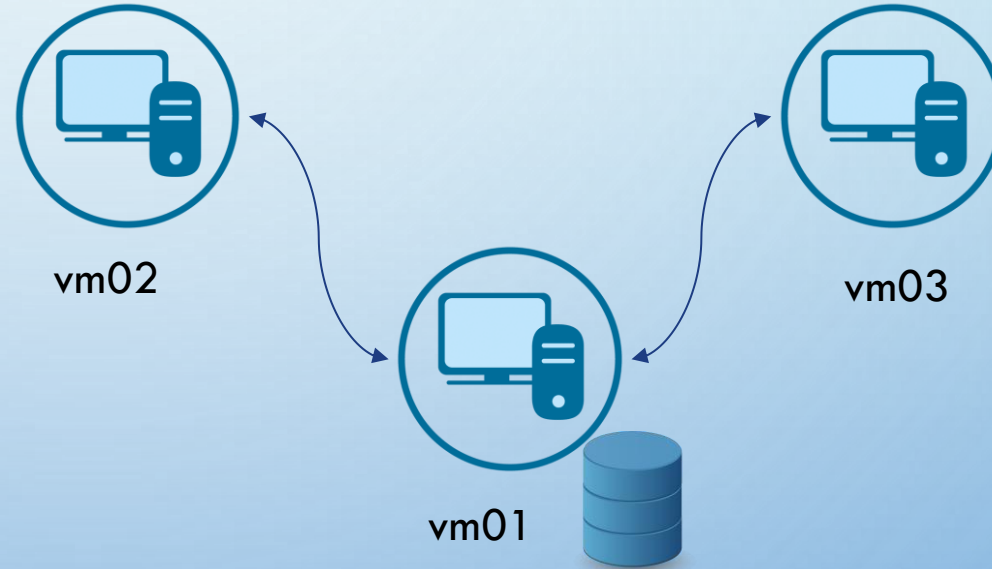
Sono stati definiti i seguenti componenti:

- **Prometheus Operator Controller Manager:** il suo scopo è il recupero delle statistiche da Prometheus
- **Database MySQL:** memorizza tutte le statistiche recuperate dall'operator
- **Scaler Operator:** implementa un opportuna logica di scaling/migrazione dei pod

SCENARIO

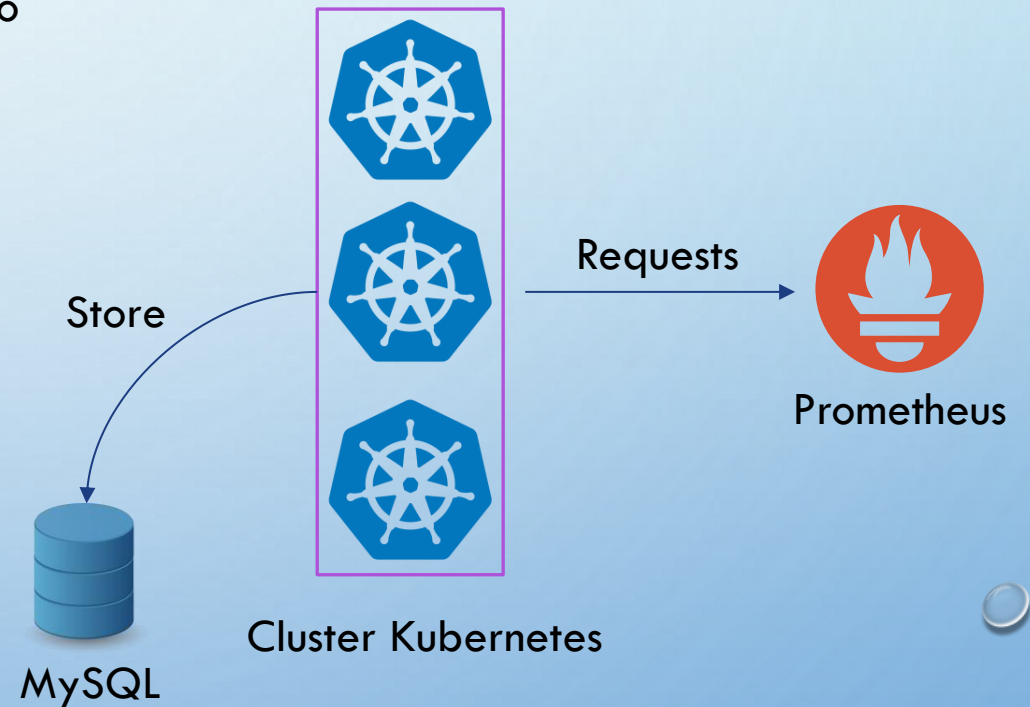
Cluster Kubernetes composto da 3 nodi:

- **Control Plane** (Vm01) → 192.168.56.11
- **Worker 1** (Vm02) → 192.168.56.12
- **Worker 2** (Vm03) → 192.168.56.13



PROMETHEUS OPERATOR CONTROLLER MANAGER

- Periodicamente (ogni 30 minuti), l'operator interroga il servizio di Prometheus per ricavare media e varianza della **CPU** e della **memoria** per ciascun pod
- Salvataggio delle statistiche in un Database MySQL



N.B. Affinché siano forniti gli opportuni privilegi per la scrittura nel DB, sono stati creati una serie di account per il range di indirizzi alla quale appartengono i vari nodi.
Una possibile miglioria è il salvataggio dell'**owner** del pod

timestamp	node_name	pod_name	avg_cpu	var_cpu	avg_mem	var_mem
2025-03-17 09:20:15	vm01	kube-controller-manager-vm01	123065	36997.4	27.9239	4065310
2025-03-17 09:20:15	vm01	kube-proxy-z7hz9	17674.6	178.822	11.319	2819460
2025-03-17 09:20:15	vm01	etcd-vm01	4479880	97037.5	25.6043	11502300
2025-03-17 09:20:15	vm01	kube-scheduler-vm01	42956.7	5418.87	13.2227	4161570
2025-03-17 09:20:15	vm01	calico-node-dnnbf	16564.4	78287.6	53.5639	103088000
2025-03-17 09:50:15	vm03	calico-kube-controllers-db6f74664-b5vq6	6053.19	1502.37	15.0812	2964.14
2025-03-17 09:50:15	vm03	coredns-8dff6757-t7v5r	53900.2	3384.71	6.48691	11.9045
2025-03-17 09:50:15	vm03	calico-node-q2rwp	26877.3	27560.6	40.4054	3328840
2025-03-17 09:50:15	vm03	kube-proxy-tqnsk	12781.7	152.257	16.7649	865.521
2025-03-17 09:50:15	vm03	scaler-operator-controller-manager-594b459f5b-lqgv7	5798.53	1001.87	4.86693	91942.6
2025-03-17 09:50:15	vm01	kube-controller-manager-vm01	140064	30391.8	28.8608	381012
2025-03-17 09:50:15	vm01	kube-proxy-z7hz9	18786.6	157.341	10.9887	45256
2025-03-17 09:50:15	vm01	etcd-vm01	4508070	81238.5	25.6668	1297430
2025-03-17 09:50:15	vm01	calico-node-dnnbf	41788.9	69214.5	59.8717	25954.3
2025-03-17 09:50:15	vm01	kube-scheduler-vm01	49253	4047.19	13.312	2408460
2025-03-17 09:50:15	vm01	kube-apiserver-vm01	13583100	850485	128.56	207039
2025-03-17 09:50:15	vm02	coredns-8dff6757-wpz5w	49994	3168.02	6.49445	1929.65
2025-03-17 09:50:15	vm02	kube-proxy-q8tjz	13559.8	181.431	16.8591	525.001
2025-03-17 09:50:15	vm02	prometheus-operator-controller-manager-5ff8849786-n9f7m	6039.56	1050.43	4.63645	90.4445
2025-03-17 09:50:15	vm02	prometheus-deployment-5dc88c6b94-4b6tg	54015	132915	93.6682	192223000
2025-03-17 09:50:16	vm02	calico-node-kgltz	25576.7	19530.8	40.2266	3645.04
2025-03-17 10:20:16	vm01	kube-controller-manager-vm01	156317	29435.4	28.7875	496436
2025-03-17 10:20:16	vm01	kube-proxy-z7hz9	19971.9	148.057	10.998	364812
2025-03-17 10:20:16	vm01	etcd-vm01	4535490	84087.3	25.6051	8591680
2025-03-17 10:20:16	vm01	calico-node-dnnbf	66137	70626.6	59.8056	108821
2025-03-17 10:20:16	vm01	kube-scheduler-vm01	55136.5	3975.53	13.6297	92605.2
2025-03-17 10:20:16	vm01	kube-apiserver-vm01	13669000	795390	127.17	5219410
2025-03-17 10:20:16	vm02	prometheus-operator-controller-manager-5ff8849786-n9f7m	8921.79	1002.03	5.07335	91159.1
2025-03-17 10:20:16	vm02	calico-node-kgltz	39088.2	20874.6	40.7727	397081
2025-03-17 10:20:16	vm02	prometheus-deployment-5dc88c6b94-4b6tg	86748.4	113832	123.115	121196000
2025-03-17 10:20:16	vm02	coredns-8dff6757-wpz5w	55220.5	2882.05	6.47918	367.999
2025-03-17 10:20:16	vm02	kube-proxy-q8tjz	14814.8	152.611	16.7201	54130.5
2025-03-17 10:20:16	vm03	kube-proxy-tqnsk	13959.2	172.648	16.8075	164.919
2025-03-17 10:20:16	vm03	scaler-operator-controller-manager-594b459f5b-lqgv7	8814.44	925.142	4.92612	93459.6
2025-03-17 10:20:16	vm03	calico-kube-controllers-db6f74664-b5vq6	9545.03	1166.32	15.1392	252.61
2025-03-17 10:20:16	vm03	coredns-8dff6757-t7v5r	59301.8	3101.57	6.49193	1.28
2025-03-17 10:20:16	vm03	calico-node-q2rwp	42207.4	29872.5	40.5604	3496780
2025-03-17 10:50:16	vm02	prometheus-deployment-5dc88c6b94-4b6tg	119799	135709	145.24	169269000
2025-03-17 10:50:16	vm02	calico-node-kgltz	52809.9	21059.5	41.0333	4292.67
2025-03-17 10:50:16	vm02	kube-proxy-q8tjz	15996.3	175.559	16.3218	1990.97
2025-03-17 10:50:16	vm02	coredns-8dff6757-wpz5w	60428.5	3242.27	6.49809	1395.88
2025-03-17 10:50:16	vm02	prometheus-operator-controller-manager-5ff8849786-n9f7m	11932	988.05	5.90626	794.235
2025-03-17 10:50:16	vm03	scaler-operator-controller-manager-594b459f5b-lqgv7	11819.9	1124.28	4.96227	101705
2025-03-17 10:50:16	vm03	calico-kube-controllers-db6f74664-b5vq6	13005.7	1387.62	15.1737	637.953
2025-03-17 10:50:16	vm03	coredns-8dff6757-t7v5r	64525.5	3465.35	6.49628	9.90476
2025-03-17 10:50:16	vm03	calico-node-q2rwp	58045.7	30194.2	40.5948	2811670
2025-03-17 10:50:16	vm03	kube-proxy-tqnsk	15167	161.489	16.8404	116.23
2025-03-17 10:50:16	vm01	kube-controller-manager-vm01	171952	28595.4	28.6821	18576.9
2025-03-17 10:50:16	vm01	kube-proxy-z7hz9	21209.5	218.712	10.9551	20946.5
2025-03-17 10:50:16	vm01	etcd-vm01	4562030	82150.8	26.5928	1865800
2025-03-17 10:50:16	vm01	calico-node-dnnbf	89994.5	64758.3	59.9119	2013740
2025-03-17 10:50:16	vm01	kube-scheduler-vm01	61080.2	4282.8	13.6923	8772.72
2025-03-17 10:50:16	vm01	kube-apiserver-vm01	13752500	849114	127.803	775615

SIMULAZIONE SCENARIO

- Collezione delle metriche significative
- Container Docker con comando **stress**
 - Parametri variabili nel tempo
 - Intervalli di tempo casuali.

```
#!/usr/bin/env python3
import time
import random
import subprocess
from datetime import datetime

def log(message):
    """ Stampa un log con timestamp """
    print(f"{datetime.now().isoformat()} - {message}", flush=True)

while True:
    # Genera valori casuali
    interval = random.randint(20, 60)
    cpu_stress = random.randint(250, 1000) # Millihertz -> da 250m a 1 GHz
    memory_stress = random.randint(300, 1000) # MB di RAM tra 300 e 1000

    log(f"Prossima esecuzione in {interval} secondi")

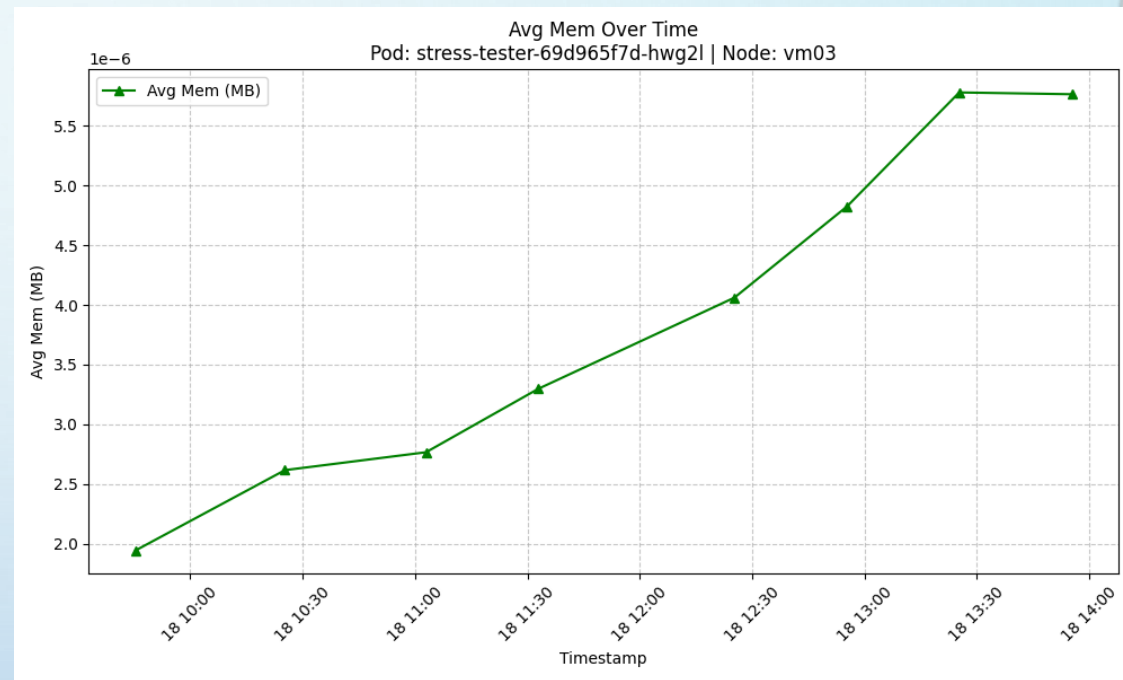
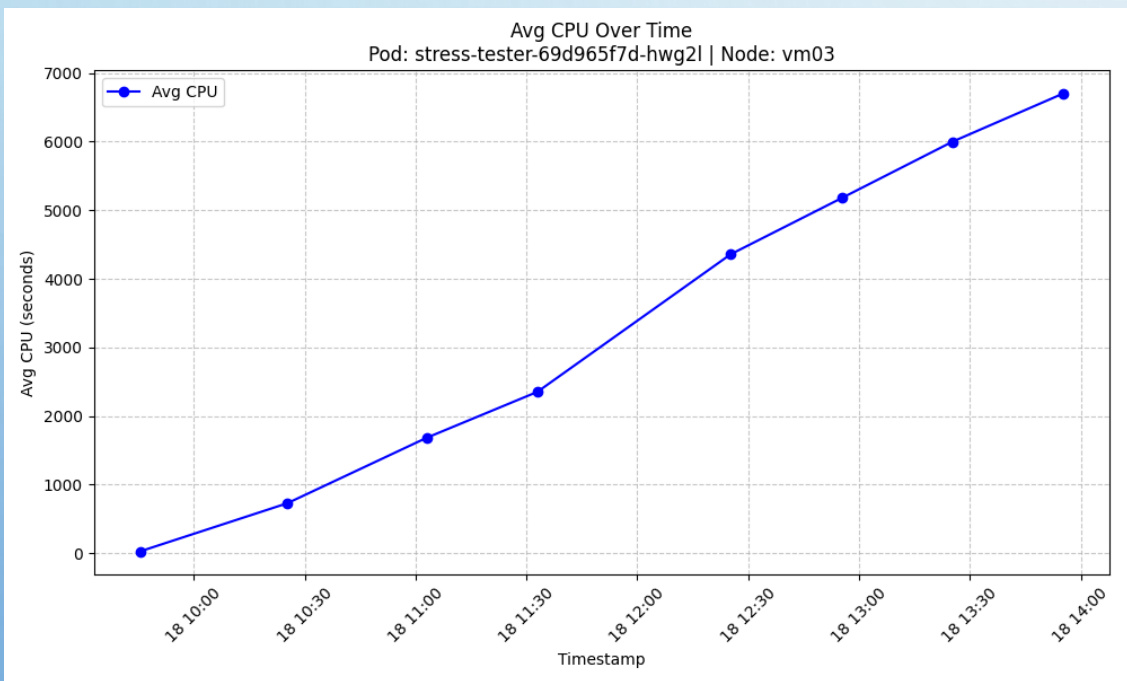
    # Attendere l'intervallo di tempo prima di eseguire stress-ng
    time.sleep(interval)

    # Lancia lo stress test
    log(f"Eseguito stress-ng con {cpu_stress} MHz CPU e {memory_stress} MB RAM")

    stress_command = [
        "stress-ng",
        "--cpu", "1", # Usiamo un core
        "--cpu-load", str(cpu_stress / 1000 * 100), # Percentuale
        "--vm", "1",
        "--vm-bytes", f"{memory_stress}M",
        "--timeout", "10s" # Test di 10 secondi
    ]

    subprocess.run(stress_command)

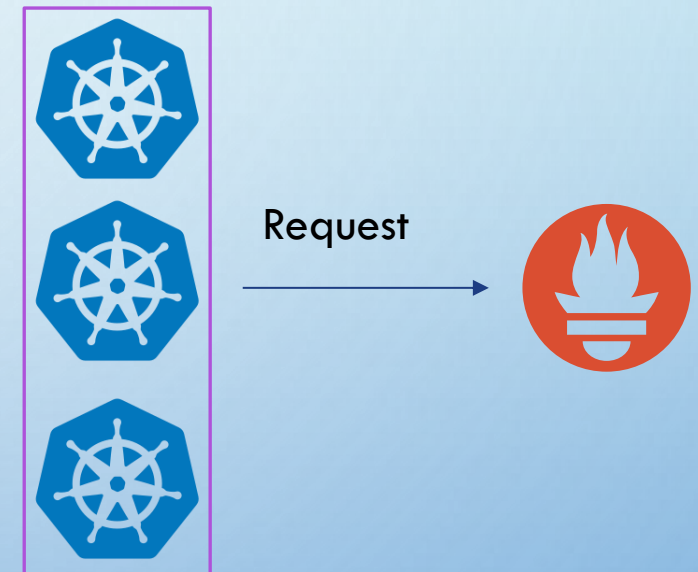
    log(f"Stress test completato. Prossima esecuzione tra {interval} secondi")
```



SCALER OPERATOR

- Periodico (periodo pari a 5 minuti),
- Si sviluppa in diverse fasi:
 1. Ricavo delle risorse libere per ciascun nodo da Prometheus
 2. Calcolo della media della CPU e della memoria libera (in %)
 3. Verifica del superamento di determinate soglie
 4. Attivazione di opportuni meccanismi di scaling e migrazione su nodi più liberi
 5. Connessione al database e estrazione metriche ricavate dal Prometheus Controller

N.B Sono applicati opportuni metodi per ricavare le risorse necessarie al pod per essere eseguite, poiché non sempre esplicitate nel pod o dall'owner.



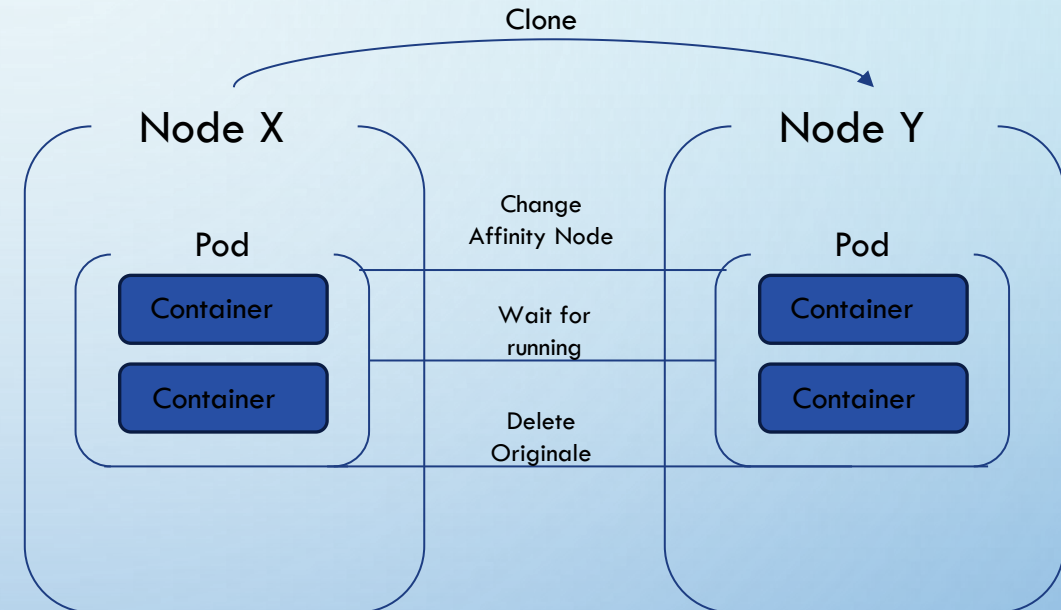
MECCANISMO DI MIGRAZIONE: POD STANDALONE

Il meccanismo di migrazione si può distinguere in:

- Pod **standalone**
- Pod **gestiti** da un controller

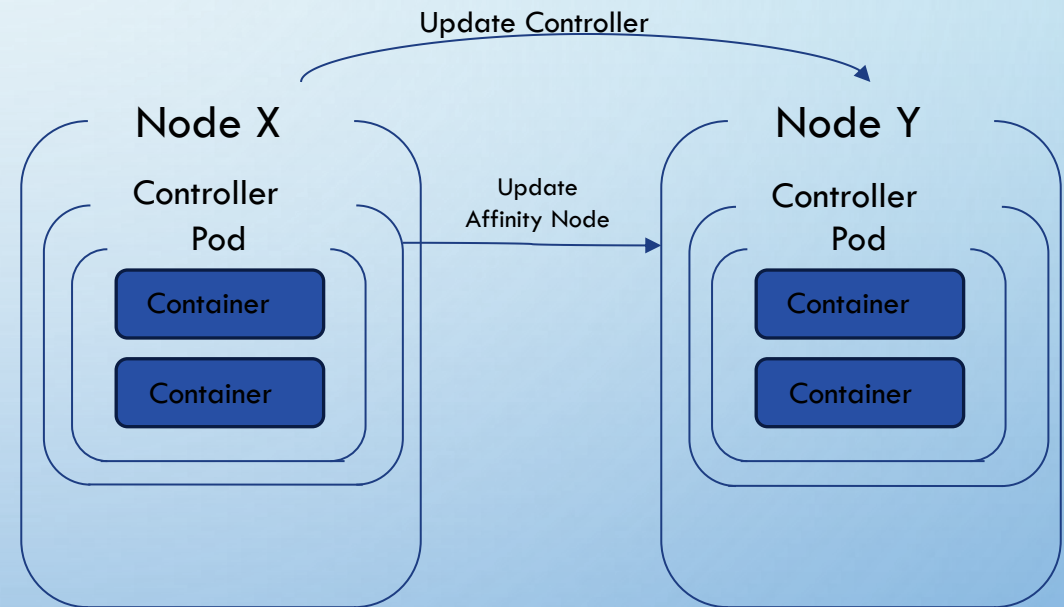
Nel primo caso:

- Si effettua una copia del Pod aggiungendo la **nodeAffinity**.
- Si elimina il Pod originale una volta che il nuovo è in fase di **running**.



MECCANISMO DI MIGRAZIONE: POD GESTITO

- Nel secondo caso si aggiorna il controller che avvia un nuovo pod con la nuova nodeAffinity e poi viene eliminato il pod originale



N.B In questo caso il controller si occupa della creazione del nuovo pod e dell'eliminazione del vecchio pod

MECCANISMI DI SCALING

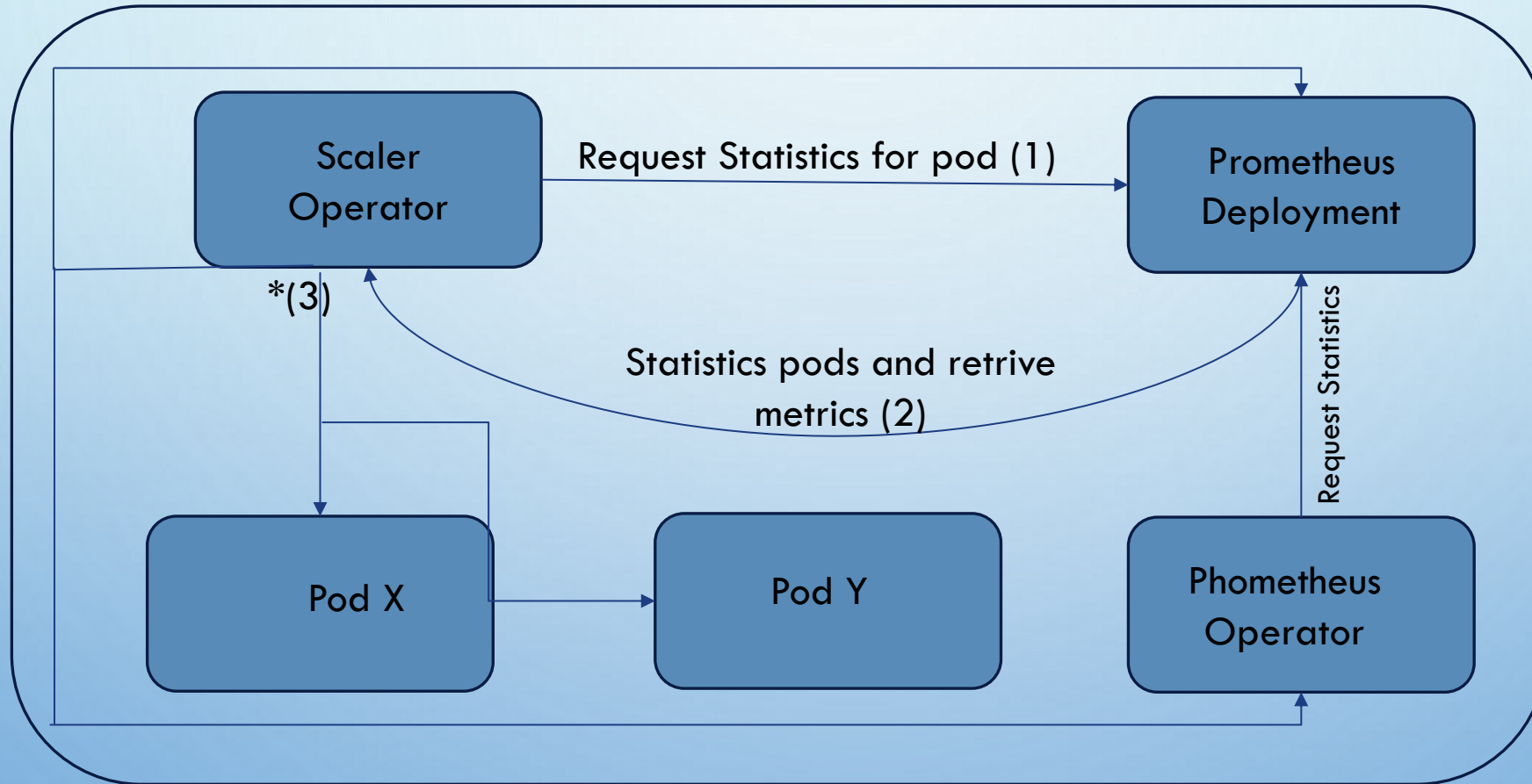
Il meccanismo di scaling si può distinguere in due casi:

1. Pod **standalone**: si effettua una copia del pod
2. Pod **gestito** da un controller: si incrementa il numero di repliche

N.B Nel secondo caso si necessita di ricavare l'owner del pod, cercando di ricavare sempre il *deploy* anche dal *replicaset*

FLUSSO COMPLESSIVO DELLO SCALER OPERATOR

Cluster Kubernetes



* Scale or Migration